

Manual: Ciência de Dados e Big Data

CRISP-DM, Machine Learning, Ensembles, Agrupamento, Avaliação, Deep Learning, IA Generativa e LLMs, Agentes, MLOps, Ética e Governança de IA, Hadoop, Spark, PLN e Python

CHANCELA EDITORIAL YLUNA85 LABS

Preparação Conjunta: Auditor Fiscal (TI) & Agente de Tributos Estaduais

Revisão Ortográfica e Glossário Geral de Siglas

Material de livre circulação interna. Revisado em Português Brasileiro (pt-BR).

1. Fundamentos de Ciência de Dados, Ciclo de Vida e Metodologia CRISP-DM

A Ciência de Dados é um campo interdisciplinar que combina computação, estatística inferencial, aprendizado de máquina e conhecimento de domínio do negócio para extrair padrões complexos e valor estratégico a partir de grandes massas de dados. Na administração tributária contemporânea, a Ciência de Dados apoia a malha fina preditiva, a detecção de notas fiscais suspeitas e a seleção de contribuintes para fiscalização com base em matrizes de risco. O padrão metodológico mais cobrado em concursos públicos é o CRISP-DM (Cross-Industry Standard Process for Data Mining), um modelo cíclico estruturado em seis fases iterativas.

- As 6 Fases do CRISP-DM

1. Entendimento do Negócio (Business Understanding - definição dos objetivos de fiscalização e conversão em metas analíticas); 2. Entendimento dos Dados (Data Understanding - coleta inicial, exploração e identificação de problemas de qualidade); 3. Preparação dos Dados (Data Preparation - seleção de tabelas, limpeza, transformação e engenharia de atributos; costuma ser a etapa que mais consome esforço do projeto, com estimativas na literatura de cerca de 70% a 80%, sem valor exato); 4. Modelagem (Modeling - seleção e aplicação de algoritmos de Machine Learning); 5. Avaliação (Evaluation - validação do modelo frente aos objetivos de negócio antes do deploy); 6. Implantação (Deployment - integração aos sistemas operacionais da SEFAZ).

- Metodologias Correlatas: KDD e SEMMA

KDD (Knowledge Discovery in Databases: Seleção, Pré-processamento, Transformação, Mineração de Dados e Interpretação/Avaliação). SEMMA (metodologia do SAS Institute: Sample, Explore, Modify, Model, Assess).

- Paradigmas de Aprendizado de Máquina

Aprendizado Supervisionado (dados rotulados com variável alvo conhecida - Classificação e Regressão), Aprendizado Não Supervisionado (dados sem rótulo prévio - Agrupamento e Associação) e Aprendizado por Reforço (agente aprende por recompensas e penalidades em um ambiente dinâmico).

ALERTA DE PROVA - O CARÁTER CÍCLICO DO CRISP-DM

O CRISP-DM NÃO é um modelo linear em cascata; ele é essencialmente iterativo e cíclico. O término de uma fase pode exigir o retorno imediato a fases anteriores (ex: a fase de Modelagem frequentemente exige retornar à fase de Preparação de Dados para refazer atributos).

2. Pré-processamento, Limpeza e Engenharia de Atributos

O pré-processamento de dados constitui a etapa preparatória determinante para o sucesso dos algoritmos de aprendizado de máquina, assegurando que os dados estejam limpos, consistentes, na escala adequada e livres de redundâncias prejudiciais.

- **Tratamento de Dados Ausentes (Missing Data)**

Exclusão de Registros (aplicável apenas quando a proporção de nulos for pequena); Imputação Estatística Univariada (substituição por Média para variáveis numéricas simétricas, Mediana para variáveis assimétricas com outliers, ou Moda para variáveis categóricas); Imputação Multivariada (KNN Imputer ou algoritmos preditivos que estimam o valor faltante com base nos vizinhos mais próximos).

- **Detecção e Tratamento de Outliers**

Identificação por Z-Score (observações com módulo de Z maior que 3) ou pelo Intervalo Interquartil (IQR): valores abaixo de Q1 menos 1,5 vezes o IQR ou acima de Q3 mais 1,5 vezes o IQR. Tratamento por winsorização (limitação aos percentis extremos), transformação logarítmica ou remoção fundamentada.

- **Escalonamento de Variáveis Numéricas**

Normalização Min-Max: leva os dados ao intervalo de 0 a 1 por $X_{norm} = (X - X_{min}) / (X_{max} - X_{min})$; sensível a outliers. Padronização Z-Score: $Z = (X - média) / desvio\ padrão$, com média zero e desvio padrão um; é adequada a algoritmos baseados em distância e em gradiente, como SVM, KNN e PCA.

- **Codificação de Variáveis Categóricas (Encoding)**

One-Hot Encoding: Cria uma nova coluna binária (0 ou 1) para cada categoria da variável nominal (ex: UF do destinatário), prevenindo que o modelo infira uma hierarquia falsa; Label/Ordinal Encoding: Atribui números inteiros sequenciais (1, 2, 3) para variáveis categóricas ordinais que possuem hierarquia intrínseca (ex: Porte da Empresa: Pequeno, Médio, Grande).

- **Redução de Dimensionalidade: PCA (Principal Component Analysis)**

Técnica não supervisionada de transformação linear ortogonal que projeta os dados em um novo sistema de coordenadas (Componentes Principais) alinhado às direções de máxima variância, calculadas a partir dos autovetores e autovalores da matriz de covariância. Elimina multicolinearidade e comprime centenas de colunas em poucas dimensões.

REGRA PRÁTICA DO ESCALONAMENTO

Algoritmos baseados em distância (KNN, K-Means, SVM) e o PCA são sensíveis à escala, e redes neurais treinam melhor com variáveis padronizadas. Árvores de decisão e os ensembles baseados em árvores (Random Forest, XGBoost) são invariantes à escala, pois dependem apenas da ordenação dos valores. Ajuste o escalonador só nos dados de treino, para evitar vazamento de dados (data leakage).

3. Aprendizado de Máquina Supervisionado: Regressão e Classificação

No aprendizado supervisionado, o modelo treina sobre variáveis preditoras (atributos, ou features, X) e uma variável alvo conhecida (rótulo Y). O objetivo é aprender uma função f que mapeia X em Y e generaliza para dados novos.

- **Modelos de Regressão Linear Regularizada**

Regressão linear com penalização dos coeficientes para reduzir overfitting: 1. Ridge (regularização L2: penaliza a soma dos quadrados dos coeficientes, multiplicada por lambda; encolhe os coeficientes sem zerá-los); 2. Lasso (regularização L1: penaliza a soma dos módulos dos coeficientes, multiplicada por lambda; pode zerar coeficientes, funcionando como seleção de variáveis); 3. ElasticNet (combinação de L1 e L2).

- **Regressão Logística**

Modelo de classificação (binária, com extensão a várias classes) que passa uma combinação linear dos atributos pela função sigmoide: $P(Y=1 | X) = 1 / (1 + e^{\text{elevado a } -(b_0 + b_1x_1 + \dots + b_nx_n)})$. Estima a probabilidade de o evento ocorrer (por exemplo, a probabilidade de uma nota fiscal ser irregular) e o resultado é comparado a um limiar de decisão.

- **K-Nearest Neighbors (KNN)**

Classificador não paramétrico baseado em instâncias ('Lazy Learner'). Não realiza treinamento prévio explícito; classifica uma nova observação com base no voto majoritário dos seus 'k' vizinhos mais próximos no espaço vetorial.

- **Naive Bayes**

Classificador probabilístico baseado no Teorema de Bayes com a premissa ingênua ('naive') de que todos os atributos são condicionalmente independentes entre si dado o rótulo da classe. Altamente eficiente para processamento de textos, análise de sentimentos e filtros de spam.

- **Support Vector Machines (SVM)**

Encontra o Hiperplano de Separação Ótimo que maximiza a margem de distância entre as classes no espaço vetorial. Utiliza Pontos de Suporte (Support Vectors) e a técnica do 'Kernel Trick' (kernels RBF, Polinomial e Sigmóide) para projetar dados não linearmente separáveis em espaços de dimensões superiores onde se tornam separáveis.

- **Árvores de Decisão (Decision Trees)**

Estrutura hierárquica com nó raiz, nós internos de decisão e folhas. A divisão dos nós usa critérios de pureza: índice de Gini (algoritmo CART) ou entropia com ganho de informação (ID3); o C4.5 usa a razão de ganho. São modelos bem interpretáveis, mas árvores muito profundas tendem ao overfitting.

DIFERENÇA CRÍTICA: RIDGE VERSUS LASSO

Lembre-se da pegadinha clássica da FGV: A regularização Lasso (L1) realiza Seleção de Atributos porque pode ZERAR completamente os coeficientes das variáveis inúteis. A regularização Ridge (L2) apenas ENCOLHE os coeficientes para valores pequenos, mantendo todas as variáveis no modelo.

4. Algoritmos de Ensemble: Bagging, Boosting e Stacking

Os métodos de Ensemble combinam as previsões de múltiplos modelos de aprendizado de máquina individuais (Modelos Fracos / Base Learners) para produzir um modelo final preditivo com desempenho superior, maior estabilidade e menor taxa de erro.

- **Bagging (Bootstrap Aggregating) e Random Forest**

Treina vários modelos independentes, em paralelo, sobre amostras aleatórias com reposição (bootstrap). A previsão final é a média (regressão) ou a votação majoritária (classificação). O Random Forest aplica o bagging a árvores de decisão e sorteia também um subconjunto de atributos em cada divisão, o que descorrelaciona as árvores e reduz a VARIÂNCIA, sem aumentar substancialmente o viés.

- **Boosting (Treinamento Sequencial Adaptativo)**

Treina modelos fracos de forma sequencial, onde cada novo modelo foca em corrigir os erros cometidos pelos modelos anteriores, atribuindo pesos maiores às observações classificadas incorretamente. Reduz primariamente o VIÉS do modelo.

- **Algoritmos de Gradient Boosting**

1. AdaBoost (ajusta pesos de amostras); 2. GBM (Gradient Boosting Machine - otimiza uma função de perda arbitrária via gradiente descendente); 3. XGBoost (Extreme Gradient Boosting - altamente otimizado para árvores com regularização integrada e processamento paralelo); 4. LightGBM (crescimento de árvores por folhas - leaf-wise, otimizado para velocidade em grandes bases); 5. CatBoost (otimizado para variáveis categóricas nativas).

- **Stacking / Blended Models**

Arquitetura hierárquica em que as previsões geradas por múltiplos modelos de base heterogêneos (ex: SVM, Random Forest e Regressão Logística) são utilizadas como features de entrada para treinar um Meta-Classificador de nível superior.

MNEMÔNICO DE ENSEMBLE

Bagging reduz Variância (combina modelos em paralelo com dados aleatórios). Boosting reduz Viés (treina modelos em série focando nos erros).

5. Aprendizado Não Supervisionado: Agrupamento e Associação

No Aprendizado Não Supervisionado, o algoritmo opera sobre dados sem rótulos ou variáveis alvo pré-definidas, com o objetivo de descobrir estruturas intrínsecas, agrupamentos naturais e regras de correlação ocultas nas informações fiscais e cadastrais.

- Agrupamento Particional: K-Means

Divide 'n' observações em 'k' clusters pré-determinados. Algoritmo iterativo: 1. Inicializa 'k' centróides aleatórios; 2. Associa cada ponto ao centróide mais próximo (distância euclidiana); 3. Recalcula os centróides como a média dos pontos do cluster; 4. Repete até a convergência. Métodos de escolha do 'k' ótimo: Método do Cotovelo (Elbow Method - ponto de inflexão da soma dos quadrados intra-cluster / WCSS) e Coeficiente de Silhueta (varia de -1 a +1; valores próximos a +1 indicam clusters densos e bem separados).

- Agrupamento Baseado em Densidade: DBSCAN

Agrupar pontos pela densidade da vizinhança, com dois parâmetros: epsilon (raio da vizinhança) e MinPts (mínimo de pontos para formar região densa). Classifica os pontos em núcleo, borda e ruído (outliers). Vantagens: não exige definir o número de clusters, encontra clusters de formato arbitrário e marca outliers como ruído. Limitações: sensível à escolha de epsilon e de MinPts e tem dificuldade com clusters de densidades muito diferentes e com dados de alta dimensão.

- Agrupamento Hierárquico

Gera uma árvore de agrupamentos, representada em um dendrograma. Aglomerativo (de baixo para cima: cada ponto começa como um cluster e os clusters são fundidos) ou divisivo (de cima para baixo). Métodos de ligação: simples (distância mínima), completa (distância máxima), média e de Ward (minimiza o aumento da variância intra-cluster).

- Regras de Associação: Algoritmo Apriori

Descobre regras do tipo "se A, então B" em transações. Suporte = proporção de transações que contêm A e B. Confiança = suporte de (A e B) dividido pelo suporte de A, ou seja, $P(B \text{ dado } A)$. Lift = $P(A \text{ e } B)$ dividido por $P(A) \times P(B)$: maior que 1 indica associação positiva; igual a 1, independência; menor que 1, associação negativa.

K-MEANS VERSUS DBSCAN

K-Means exige definir k, assume clusters aproximadamente esféricos e é sensível a outliers. O DBSCAN não exige k, encontra formatos arbitrários e trata outliers como ruído, mas depende de epsilon e MinPts e sofre com densidades muito variadas.

6. Avaliação de Modelos, Matriz de Confusão e Curva ROC

A avaliação criteriosa de modelos preditivos é indispensável para mensurar sua capacidade de generalização e evitar os fenômenos de Underfitting e Overfitting. A escolha da métrica correta deve refletir o custo operacional do erro no contexto da fiscalização tributária.

- **O Trade-off Viés-Variância (Bias-Variance Tradeoff)**

Underfitting (Alto Viés / Baixa Variância): O modelo é excessivamente simplista para capturar a complexidade dos dados, apresentando alto erro tanto no treino quanto no teste. Overfitting (Baixo Viés / Alta Variância): O modelo decora o conjunto de treino (incluindo ruídos), apresentando excelente desempenho no treino e péssimo desempenho no teste.

- **Validação Cruzada (Cross-Validation)**

K-Fold: divide os dados em k partes, treina em k-1 e testa na restante, repetindo k vezes e tirando a média. K-Fold estratificado: preserva a proporção das classes em cada parte, recomendado para bases desbalanceadas. Em séries temporais, não se embaralha: a validação respeita a ordem do tempo (treino no passado, teste no futuro).

- **Matriz de Confusão para Classificação Binária**

Quatro quadrantes fundamentais: Verdadeiro Positivo (VP - Fraude identificada corretamente), Verdadeiro Negativo (VN - Contribuinte regular classificado corretamente), Falso Positivo (FP / Erro Tipo I - Contribuinte honesto apontado incorretamente como fraudador), Falso Negativo (FN / Erro Tipo II - Fraude real que passou despercebida pelo sistema).

- **Métricas de Classificação**

Acurácia = $(VP + VN) / \text{total}$ (enganosa em bases desbalanceadas). Precisão = $VP / (VP + FP)$: dos alertas, quantos eram reais. Revocação (recall, sensibilidade) = $VP / (VP + FN)$: das ocorrências reais, quantas foram capturadas. Especificidade = $VN / (VN + FP)$. F1-Score = $2 \times \text{Precisão} \times \text{Revocação} / (\text{Precisão} + \text{Revocação})$, a média harmônica das duas. Em bases muito desbalanceadas, a curva Precisão-Revocação e sua área costumam ser mais informativas que a curva ROC.

- **Curva ROC e AUC**

A curva ROC plota a taxa de verdadeiros positivos (sensibilidade), no eixo Y, contra a taxa de falsos positivos (1 menos a especificidade), no eixo X, para vários limiares de decisão. A AUC (área sob a curva) vai de 0,5, equivalente a um classificador aleatório, a 1,0, classificador perfeito (valores abaixo de 0,5 indicam desempenho pior que o acaso).

A ARMADILHA DA ACURÁCIA EM FRAUDES FISCAIS

Em uma base onde 99% das notas fiscais são idôneas e 1% são fraudulentas, um modelo ingênuo que classifique TODAS as notas como 'idôneas' terá 99% de Acurácia, mas terá 0% de Recall e será completamente inútil para o Fisco!

7. Arquitetura de Big Data e o Ecossistema Apache Hadoop

Big Data refere-se a conjuntos de dados cujo volume, velocidade de geração e variedade de formatos inviabilizam o armazenamento e processamento por sistemas de bancos de dados relacionais tradicionais. A gestão contemporânea de Big Data assenta-se no paradigma de computação distribuída em clusters de hardware comum (Commodity Hardware).

- **Os 5 Vs do Big Data**

Volume (escala de petabytes a exabytes), Velocidade (fluxos de dados gerados e processados em tempo real), Variedade (dados estruturados como tabelas SQL, semiestruturados como XML/JSON e não estruturados como imagens e vídeos), Veracidade (qualidade, autenticidade e confiabilidade da fonte) e Valor (relevância dos insights para as decisões estratégicas do Fisco).

- **HDFS (Hadoop Distributed File System)**

Arquitetura mestre e trabalhadores: 1. NameNode: gerencia o namespace e os metadados (mantidos em memória) e sabe em quais DataNodes estão os blocos; 2. DataNodes: armazenam os blocos no disco local; 3. Replicação: cada bloco (tamanho padrão de 128 MB) é replicado, por padrão, em 3 nós, com recuperação automática de falhas.

- **YARN (Yet Another Resource Negotiator)**

Camada de gerenciamento de recursos e escalonamento de tarefas do Hadoop, composta pelo ResourceManager central e NodeManagers locais em cada nó do cluster.

- **O Paradigma de Programação MapReduce**

Processamento em lote distribuído estruturado em três fases principais: 1. Fase Map: Aplica uma função de transformação sobre os dados de entrada, gerando pares intermediários chave-valor (chave, valor); 2. Fase Shuffle and Sort: Agrupa e ordena automaticamente todos os valores associados à mesma chave intermediária; 3. Fase Reduce: Aplica uma função agregadora sobre os valores de cada chave, gerando o resultado consolidado final.

ALERTA DE PROVA - NAMENODE

O NameNode do HDFS armazena apenas METADADOS (estrutura de diretórios, permissões e mapeamento de quais blocos estão em quais DataNodes). O NameNode NUNCA armazena os blocos de dados dos arquivos dos usuários.

8. Processamento Distribuído em Memória com Apache Spark

O Apache Spark é um motor unificado de processamento de dados em larga escala. Ele reduz o gargalo de leitura e escrita em disco do MapReduce ao manter dados intermediários em memória. O projeto Spark informa, em seus benchmarks, ganhos de até cerca de 100 vezes em memória e 10 vezes em disco sobre o MapReduce; o ganho real depende da carga de trabalho.

- **Arquitetura do Apache Spark**

Driver (coordena a execução, cria a SparkSession e o plano de execução como DAG), gerenciador de cluster (YARN, Kubernetes ou o modo Standalone; o suporte ao Mesos foi depreciado) e executors (processos nos nós de trabalho que executam as tarefas e mantêm dados em cache).

- **RDD (Resilient Distributed Datasets)**

Abstração fundamental do Spark: Coleção distribuída de objetos imutável, particionada e tolerante a falhas por meio de Linhagem (Lineage Graph - capacidade de reconstruir qualquer partição perdida reexecutando as transformações originais).

- **Operações em RDD: Transformações versus Ações**

Transformações: Criam um novo RDD a partir de um RDD existente (ex: map, filter, flatMap, groupByKey, reduceByKey, join). Operam sob o conceito de Execução Tardia (Lazy Evaluation) - não são executadas imediatamente, apenas registradas no DAG; Ações: Disparam a execução física do plano de computação do DAG e retornam um valor ao Driver ou gravam dados no armazenamento (ex: count, collect, reduce, first, take, saveAsTextFile).

- **Módulos Integrados do Ecossistema Spark**

Spark SQL (DataFrames e Datasets com otimizador de consultas Catalyst e execução Tungsten), Spark Streaming / Structured Streaming (processamento contínuo de fluxos de eventos em tempo real), Spark MLlib (biblioteca de algoritmos de Machine Learning distribuído) e GraphX (processamento de grafos).

PEGADINHA CLÁSSICA DE SPARK DA FGV

Transformações no Spark NUNCA processam dados no momento da chamada; elas são apenas adicionadas ao plano de execução (Lazy Evaluation). O processamento de fato só ocorre quando uma AÇÃO (como count() ou collect()) é invocada!

9. Processamento de Linguagem Natural e Mineração de Grafos Fiscais

Técnicas avançadas de Inteligência Artificial voltadas para a análise de dados textuais não estruturados (descrições de produtos em NF-e, contratos sociais, defesas de autos de infração) e redes complexas de relacionamentos societários e financeiros.

- **Pipeline de Processamento de Linguagem Natural (PLN)**

1. Tokenização (dividir o texto em tokens); 2. Remoção de stopwords (palavras muito frequentes e pouco informativas, como artigos e preposições); 3. Normalização léxica: stemming (corta sufixos por regras, gerando radicais que podem não existir no dicionário; o algoritmo de Porter é para o inglês, e para o português há stemmers como o RSLP) versus lematização (retorna o lema, a forma de dicionário, com apoio de análise morfológica).

- **Vetorização Textual: TF-IDF versus Embeddings**

TF-IDF pondera a frequência do termo no documento pelo inverso de sua frequência no corpus, destacando termos característicos. Embeddings são vetores densos de números reais em que significados semelhantes ficam próximos. Word2Vec e FastText geram embeddings estáticos (um vetor por palavra); modelos como o BERT geram embeddings contextuais (o vetor depende da frase).

- **Mineração de Grafos na Fiscalização Tributária**

Modelagem de dados em Vértices (empresas, sócios, procuradores, contas bancárias) e Arestas (vínculos societários, transferências financeiras, emissões de notas fiscais). Detecção de comunidades (Algoritmo de Louvain), medidas de Centralidade de Grau, Proximidade e Intermediação (Betweenness Centrality - identifica o operador central de esquemas de lavagem de dinheiro), e algoritmos de detecção de ciclos para coibir operações triangulares de crédito fictício de ICMS.

STEMMING VERSUS LEMATIZAÇÃO

Stemming corta o final da palavra e pode gerar radicais inexistentes (por exemplo, "estudante" para "estud"). Lematização usa o vocabulário e a morfologia para devolver o lema real (por exemplo, "estudando" para "estudar" e "melhores" para "melhor").

10. Armadilhas Conceituais Recorrentes em Ciência de Dados e Big Data

Pontos conceituais que costumam gerar erro em questões de ciência de dados, aprendizado de máquina e Big Data. Servem de revisão rápida; a base de cada ponto está nas seções anteriores.

- **Pegadinha 1: Lazy Evaluation no Spark**
Chamar ``rdd.map()`` ou ``rdd.filter()`` não consome CPU nem lê dados imediatamente; a execução real só inicia quando uma ação (``count()``, ``collect()``) é disparada.
- **Pegadinha 2: NameNode armazena Metadados, não Dados**
O NameNode do HDFS armazena exclusivamente a árvore de metadados em memória RAM. Os blocos de dados dos arquivos residem nos DataNodes.
- **Pegadinha 3: Lasso (L1) Zera Coeficientes, Ridge (L2) Não**
Lasso é seleção de variáveis porque zera os coeficientes de atributos irrelevantes. Ridge apenas encolhe os coeficientes.
- **Pegadinha 4: Acurácia em Bases Desbalanceadas**
Acurácia alta em dados desbalanceados é ilusória. Para detecção de fraudes fiscais raras, as métricas mandatórias são Recall, Precisão e F1-Score.
- **Pegadinha 5: K-Means Exige Definição Prévia de 'k'**
O K-Means não descobre o número de clusters sozinho; o usuário é obrigado a fornecer o parâmetro 'k'. O DBSCAN, por outro lado, descobre o número de clusters automaticamente.
- **Pegadinha 6: Árvores e Ensembles de Árvores são Invariantes à Escala**
Random Forest e Gradient Boosting não exigem normalização ou padronização de variáveis numéricas, pois usam a ordenação dos valores. Já KNN, K-Means, SVM e PCA são sensíveis à escala.
- **Pegadinha 7: Premissa Ingênua do Naive Bayes**
O Naive Bayes assume independência CONDICIONAL entre todos os atributos dada a classe, o que raramente é verdade na prática, mas ainda assim produz excelentes classificações.
- **Pegadinha 8: AUC de um Classificador Aleatório**
Um modelo que decide por acaso tem AUC de cerca de 0,50 (a diagonal da curva ROC). O classificador perfeito tem AUC igual a 1,00.
- **Pegadinha 9: Underfitting é Alto Viés, Overfitting é Alta Variância**
Underfitting decorre de modelo fraco/simples. Overfitting decorre de modelo hipercomplexo que memoriza o ruído do treino.
- **Pegadinha 10: RDDs são Imutáveis**
No Apache Spark, RDDs são coleções 100% imutáveis. Qualquer transformação gera um NOVO RDD, mantendo o grafo de linhagem original intacto para recuperação de falhas.

SÍNTESE FINAL DE MEMORIZAÇÃO

Lembre-se: Spark transforma preguiçosamente (lazy); Lasso zera (L1); em fraudes raras olhe Recall e F1, nunca Acurácia; e no HDFS o NameNode só guarda metadados!

11. Outros Paradigmas: Semissupervisionado, Autossupervisionado, por Reforço e Transferência

Além do aprendizado supervisionado e do não supervisionado, o programa cita outros paradigmas.

- **Aprendizado semissupervisionado**

Usa poucos dados rotulados e muitos não rotulados. Exemplos de técnicas: autotreinamento (self-training, o modelo rotula com alta confiança e reaprende) e propagação de rótulos em grafos. Útil quando rotular é caro.

- **Aprendizado autossupervisionado**

Os rótulos vêm dos próprios dados, sem anotação humana: por exemplo, prever uma palavra mascarada em uma frase ou o próximo token. É a base do pré-treinamento de grandes modelos de linguagem.

- **Aprendizado por reforço**

Um agente interage com um ambiente: observa o estado, escolhe uma ação, recebe uma recompensa e busca uma política que maximize a recompensa acumulada. Conceitos: processo de decisão de Markov, função de valor, Q-learning e o dilema exploração versus exploração (explorar ações novas ou usar a melhor conhecida).

- **Aprendizado por transferência**

Reaproveita um modelo já treinado em uma tarefa ou base grande e o ajusta (fine-tuning) para outra tarefa, com menos dados e menos custo.

- **Aprendizado ativo**

O modelo escolhe quais exemplos não rotulados devem ser rotulados por um especialista, priorizando os mais informativos.

QUEM FORNECE O RÓTULO?

Supervisionado: rótulos humanos para todos. Semissupervisionado: só para parte. Autossupervisionado: gerados dos próprios dados. Não supervisionado: sem rótulos. Por reforço: recompensas do ambiente.

12. Redes Neurais e Deep Learning

Redes neurais artificiais são modelos formados por camadas de unidades (neurônios) que combinam entradas com pesos e aplicam uma função não linear. Deep learning usa redes com muitas camadas.

- **Neurônio artificial e funções de ativação**

Cada neurônio calcula a soma ponderada das entradas mais um viés (bias) e aplica uma função de ativação: sigmoide (saída de 0 a 1), tanh (-1 a 1), ReLU (zero para valores negativos, identidade para positivos; muito usada em camadas ocultas) e softmax (transforma um vetor em probabilidades que somam 1; usada na saída de classificação multiclasse).

- **Treinamento**

Função de perda mede o erro (erro quadrático médio em regressão; entropia cruzada em classificação). A retropropagação (backpropagation) calcula os gradientes e o gradiente descendente (em variantes como SGD, mini-lotes e Adam) atualiza os pesos. Uma época é uma passagem completa pelos dados de treino.

- **Combate ao overfitting**

Regularização L1 e L2, dropout (desliga neurônios aleatoriamente no treino), parada antecipada (early stopping), aumento de dados (data augmentation) e normalização em lote (batch normalization). O problema do gradiente que desaparece (vanishing gradient) afeta redes profundas com certas ativações.

- **Arquiteturas**

MLP (camadas totalmente conectadas); CNN (convoluções e pooling; imagens); RNN, LSTM e GRU (sequências; o LSTM usa portas de esquecimento, entrada e saída para reter informação de longo prazo); Transformers (autoatenção, base de LLMs); autoencoders (compressão e reconstrução); GANs (gerador e discriminador em disputa); modelos de difusão (geram dados removendo ruído gradualmente).

- **Transformers**

Apresentados no artigo "Attention Is All You Need" (2017). Usam autoatenção para relacionar todas as posições de uma sequência em paralelo, sem recorrência, o que escala bem em GPUs e substituiu as RNNs na maior parte das tarefas de linguagem.

- **Hardware**

GPUs e TPUs aceleram as multiplicações de matrizes do treinamento. Modelos maiores costumam melhorar o desempenho, mas isso depende dos dados e da arquitetura e tem custo alto.

DESEMPENHO NÃO É INDEPENDENTE DO TAMANHO DO MODELO

Aumentar ou reduzir o modelo muda a capacidade e o risco de overfitting. Não há garantia de que a precisão seja a mesma independentemente do tamanho da rede, inclusive em LSTM.

13. IA Generativa e Grandes Modelos de Linguagem (LLMs)

IA generativa cria conteúdo novo (texto, imagem, código, áudio). Os grandes modelos de linguagem (LLMs) são redes Transformer treinadas em enormes volumes de texto.

- **Como funcionam**

O texto é dividido em tokens, convertidos em embeddings e processados por camadas de autoatenção. No pré-treinamento, o modelo aprende a prever o próximo token. A geração produz um token por vez, amostrando da distribuição de probabilidades.

- **Ajuste do modelo**

Ajuste fino supervisionado (fine-tuning) e ajuste por instruções treinam o modelo para seguir comandos. O aprendizado por reforço a partir de feedback humano (RLHF) alinha as respostas às preferências humanas. LoRA é uma técnica de ajuste eficiente em parâmetros que treina matrizes pequenas em vez de todos os pesos.

- **Prompt e parâmetros de geração**

Engenharia de prompt orienta o modelo por instruções, exemplos (few-shot) e contexto. Aprendizado em contexto (in-context learning) é a capacidade de seguir um padrão dado no próprio prompt. Temperatura maior aumenta a aleatoriedade; top-p e top-k limitam os tokens candidatos. A janela de contexto limita o tamanho da entrada.

- **RAG (geração aumentada por recuperação)**

Busca trechos relevantes em uma base de conhecimento (por similaridade de embeddings em um banco vetorial) e os inclui no prompt, para que o modelo responda com base neles. Reduz alucinações e permite dados atualizados, mas não os elimina.

- **Limitações e riscos**

Alucinações (respostas plausíveis e falsas), vieses dos dados, vazamento de dados sensíveis, injeção de prompt e resultados não determinísticos. Exigem validação humana em decisões relevantes, controles de acesso e conformidade com a LGPD.

- **Avaliação**

Perplexidade (para modelos de linguagem), métricas de sobreposição como BLEU e ROUGE (tradução e resumo), benchmarks de tarefas e avaliação humana. Nenhuma métrica isolada captura a qualidade geral.

RAG NÃO É RETREINAR O MODELO

No RAG, o modelo não é retreinado: a informação externa é recuperada e incluída no contexto a cada consulta. Ajuste fino, ao contrário, altera os pesos do modelo.

14. Agentes Inteligentes e Sistemas Multiagentes

Um agente é uma entidade que percebe o ambiente por sensores e age sobre ele por atuadores, buscando atingir objetivos. Um agente racional escolhe a ação que maximiza o desempenho esperado (Russell e Norvig).

- **Tipos de agentes (Russell e Norvig)**

Reativo simples (age só pela percepção atual, por regras), reativo baseado em modelo (mantém um estado interno do ambiente), baseado em objetivos (escolhe ações que levam ao objetivo), baseado em utilidade (maximiza uma função de utilidade) e agente com aprendizado (melhora com a experiência).

- **Propriedades do ambiente**

Totalmente ou parcialmente observável; determinístico ou estocástico; episódico ou sequencial; estático ou dinâmico; discreto ou contínuo; agente único ou multiagente.

- **Sistemas multiagentes**

Vários agentes autônomos que interagem em um ambiente comum. Aspectos: comunicação (por exemplo, linguagens de comunicação de agentes, como a FIPA ACL), coordenação, cooperação, competição e negociação. Aplicam-se a simulação, logística e automação distribuída.

- **Agentes baseados em LLM**

Combinam um modelo de linguagem com planejamento, memória e uso de ferramentas (chamada de funções e APIs) em um laço de raciocínio e ação; o padrão ReAct alterna raciocínio e ações. Protocolos como o MCP (Model Context Protocol) padronizam a conexão de modelos a ferramentas e dados.

- **Riscos de agentes**

Agência excessiva (permissões maiores que o necessário), injeção de prompt por conteúdo externo e ações irreversíveis. Mitigações: privilégio mínimo, aprovação humana para ações críticas, registro e limites de execução (veja a apostila de Segurança).

REATIVO SIMPLES NÃO TEM MEMÓRIA

O agente reativo simples decide só pela percepção atual. Para ambientes parcialmente observáveis, é preciso um agente com modelo interno (estado).

15. MLOps: do Notebook à Produção

MLOps aplica práticas de DevOps ao ciclo de vida de modelos de aprendizado de máquina, para implantá-los e mantê-los de forma confiável e reprodutível.

- **O que MLOps acrescenta ao DevOps**

Além do código, versiona dados e modelos; automatiza o treinamento; monitora a qualidade do modelo após a implantação. Além de CI e CD, inclui o treinamento contínuo (CT), que retreina o modelo quando necessário.

- **Componentes**

Versionamento de código, dados e modelos; rastreamento de experimentos (por exemplo, MLflow); repositório de atributos (feature store); pipelines de treinamento, validação e implantação; registro de modelos (model registry); contêineres e orquestração (Docker, Kubernetes e Kubeflow).

- **Níveis de maturidade (Google)**

Nível 0: processo manual. Nível 1: automação do pipeline de ML (treinamento contínuo). Nível 2: automação de CI/CD do pipeline.

- **Formas de implantação**

Lote (batch): previsões periódicas. Online: modelo exposto por API (REST ou gRPC), com baixa latência. Fluxo (streaming): previsões sobre eventos contínuos. Na borda (edge): no dispositivo.

- **Estratégias de liberação**

Canário (uma fração do tráfego), azul-verde (duas versões, com troca rápida), sombra (a nova versão recebe cópia do tráfego sem afetar respostas) e teste A/B.

- **Monitoramento e deriva**

Deriva de dados (data drift: a distribuição das entradas muda), deriva de conceito (concept drift: a relação entre entradas e alvo muda) e queda de desempenho. Também monitorar latência, erros e uso de recursos. Limiares definem quando retreinar.

- **Serviços de nuvem e documentação**

Plataformas gerenciadas como Amazon SageMaker, Azure Machine Learning e Google Vertex AI. Cartões de modelo (model cards) e documentação de conjuntos de dados registram uso pretendido, limitações e métricas.

MODELO EM PRODUÇÃO ENVELHECE

Os dados do mundo mudam. Sem monitoramento de deriva e retreino, um modelo bom no dia da implantação perde qualidade com o tempo.

16. Ética, Explicabilidade e Governança de IA

A IA aplicada ao setor público exige transparência, justiça e responsabilidade, especialmente quando afeta direitos de contribuintes.

- **Interpretabilidade e explicabilidade**

Interpretabilidade: o quanto um humano entende o funcionamento do modelo (árvores e regressões são interpretáveis).
Explicabilidade: métodos que explicam decisões de modelos complexos, como LIME e SHAP (explicações locais, por previsão), importância de atributos e gráficos de dependência parcial.

- **Viés algorítmico**

Pode vir dos dados (amostras não representativas, rótulos enviesados), da modelagem e do uso. Métricas de justiça incluem paridade demográfica, igualdade de oportunidades e probabilidades equalizadas; em geral não é possível satisfazer todas ao mesmo tempo.

- **LGPD e decisões automatizadas**

O art. 20 da LGPD assegura ao titular o direito de solicitar a revisão de decisões tomadas unicamente com base em tratamento automatizado de dados pessoais que afetem seus interesses, e o direito a informações claras sobre os critérios e procedimentos, observados os segredos comercial e industrial.

- **Referências regulatórias e normativas**

NIST AI Risk Management Framework 1.0 (funções Governar, Mapear, Medir e Gerenciar); ISO/IEC 42001:2023 (sistema de gestão de IA); Regulamento (UE) 2024/1689, o AI Act europeu, com abordagem baseada em risco (risco inaceitável, alto, limitado e mínimo). No Brasil, o Projeto de Lei nº 2.338/2023, sobre IA, está em tramitação: conferir a situação atual.

- **Boas práticas**

Transparência sobre o uso de IA, supervisão humana em decisões relevantes, testes de viés, documentação de dados e modelos, registro de decisões (auditabilidade) e minimização de dados pessoais.

EXPLICAR E PODER CONTESTAR

Em decisões automatizadas que afetam o contribuinte, a explicabilidade e a possibilidade de revisão (LGPD, art. 20) são requisitos, não opcionais.

17. Avaliação de Regressão e Boas Práticas de Modelagem

Complementa a seção 6 com métricas de regressão e cuidados para modelos confiáveis.

- **Métricas de regressão**

MAE (erro absoluto médio), MSE (erro quadrático médio), RMSE (raiz do MSE, na mesma unidade do alvo, penaliza mais os erros grandes) e R^2 (proporção da variância explicada pelo modelo, de 0 a 1 em casos usuais; R^2 ajustado penaliza atributos inúteis).

- **Pressupostos da regressão linear**

Linearidade, independência dos erros, homocedasticidade (variância constante dos resíduos) e normalidade dos resíduos (importante para inferência). Multicolinearidade entre preditores prejudica a interpretação dos coeficientes.

- **Divisão dos dados e vazamento**

Treino, validação (ajuste de hiperparâmetros) e teste (avaliação final, usado uma vez). Vazamento de dados (data leakage) ocorre quando informação do teste ou do futuro vaza para o treino, inflando o desempenho; por exemplo, escalonar antes de dividir.

- **Desbalanceamento de classes**

Reamostragem (subamostragem da classe majoritária, sobreamostragem da minoritária, SMOTE, que sintetiza exemplos), pesos de classe, ajuste do limiar de decisão e métricas apropriadas (Precisão, Revocação, F1, PR-AUC).

- **Ajuste de hiperparâmetros**

Busca em grade (grid search), busca aleatória (random search) e otimização bayesiana, sempre com validação cruzada. Hiperparâmetros são definidos antes do treino; parâmetros (como pesos) são aprendidos.

- **Seleção de atributos**

Métodos de filtro (estatísticas univariadas), de envelope (wrapper, como eliminação recursiva) e embutidos (como Lasso e importância em árvores).

TESTE SÓ NO FINAL

O conjunto de teste não deve orientar decisões de modelagem. Se for reutilizado para escolher modelos, deixa de estimar bem o desempenho real.

18. Hadoop e Spark: Complementos

Complementa as seções 7 e 8 com pontos frequentes sobre o ecossistema Hadoop e o Spark.

- **Alta disponibilidade do HDFS**
O NameNode é um ponto único de falha em configuração simples. Com alta disponibilidade, há um NameNode ativo e um em espera (standby), com JournalNodes para compartilhar o log de edições. O Secondary NameNode não é um backup em espera: ele apenas faz pontos de verificação (checkpoints) dos metadados.
- **Codificação de eliminação (Hadoop 3)**
A replicação tripla consome 200% de espaço extra. A codificação de eliminação (erasure coding) protege os dados com paridade e tem menor custo de armazenamento, ao custo de mais processamento.
- **YARN em detalhe**
ResourceManager (escalonador e gestor de aplicações), NodeManager (por nó), ApplicationMaster (por aplicação, negocia recursos) e containers (unidades de recursos onde as tarefas executam).
- **MapReduce: Combiner e Partitioner**
O combiner faz uma agregação local antes do shuffle, reduzindo tráfego de rede. O partitioner define a qual reducer cada chave vai (por padrão, por hash da chave).
- **Ferramentas do ecossistema Hadoop**
Hive (consultas em SQL sobre dados no HDFS, com HiveQL), HBase (banco NoSQL colunar sobre HDFS), Pig (scripts de fluxo de dados), Sqoop (transferência entre Hadoop e bancos relacionais), Flume (ingestão de logs), Oozie (orquestração de fluxos) e ZooKeeper (coordenação distribuída).
- **Spark: transformações estreitas e largas**
Estreitas (map, filter): cada partição de saída depende de uma partição de entrada, sem shuffle. Largas (reduceByKey, groupByKey, join): exigem shuffle entre partições, definindo fronteiras de estágios do DAG. reduceByKey agrega localmente antes do shuffle e tende a ser preferível ao groupByKey.
- **Spark: DataFrames, cache e desempenho**
DataFrames e Datasets têm esquema e passam pelo otimizador Catalyst; RDDs não. cache() e persist() mantêm dados em memória para reutilização. Junção por broadcast envia uma tabela pequena a todos os executors. A execução adaptativa de consultas (AQE) ajusta o plano em tempo de execução. Assimetria de dados (data skew) causa tarefas lentas.

SECONDARY NAMENODE NÃO É RESERVA

Pegadinha clássica: o Secondary NameNode não assume o lugar do NameNode em caso de falha. A alta disponibilidade usa um NameNode standby.

19. Python para Ciência de Dados: NumPy, pandas e scikit-learn

Python é a linguagem mais usada em ciência de dados, com um ecossistema de bibliotecas.

- **NumPy**
Arrays multidimensionais homogêneos (ndarray) com operações vetorizadas, mais rápidas que laços em Python. Atributos: shape (dimensões), dtype (tipo) e ndim. Broadcasting permite operar arrays de formas compatíveis.
- **pandas**
Series (uma dimensão) e DataFrame (tabela com linhas e colunas rotuladas). Operações comuns: head(), info(), describe(), groupby(), merge() (junções), fillna() e dropna() (ausentes), apply() e leitura de arquivos (read_csv, read_parquet). O atributo shape de um DataFrame retorna a tupla (linhas, colunas).
- **Seleção em pandas**
loc seleciona por rótulos e iloc por posições inteiras. Máscaras booleanas filtram linhas (df[df["valor"] > 100]).
- **scikit-learn**
API consistente: fit() treina, predict() prediz e transform() transforma; fit_transform() combina os dois. Pipeline encadeia pré-processamento e modelo, evitando vazamento. Módulos para divisão de dados, validação cruzada, métricas e busca de hiperparâmetros.
- **Visualização e ambiente**
Matplotlib, seaborn e plotly para gráficos. Jupyter Notebook para exploração interativa. PySpark leva a API do Spark para Python e usa DataFrames distribuídos.
- **Boas práticas**
Ambientes virtuais e arquivos de dependências para reprodutibilidade; fixar sementes aleatórias; código de pipelines fora de notebooks quando for para produção.

SHAPE DEVOLVE TUPLA, SIZE DEVOLVE TOTAL

Em NumPy e pandas, shape devolve as dimensões (linhas, colunas), e size, o número total de elementos (linhas vezes colunas).

20. Aprofundamento: RAG, Agentes e Métricas de Avaliação de Modelos

Dois temas concentram a maior atualização recente do programa de Ciência de Dados e IA: a arquitetura RAG (Retrieval-Augmented Generation) para reduzir alucinações de LLMs, e as métricas formais para avaliar a qualidade de um modelo, tema clássico e ainda mais cobrado desde que a IA generativa entrou nos editais.

- **Arquitetura RAG (Retrieval-Augmented Generation)**

Combina um LLM com uma etapa de recuperação de informação: a pergunta do usuário é usada para buscar trechos relevantes em uma base de conhecimento (geralmente indexada por embeddings em um banco vetorial), e esses trechos são injetados no prompt enviado ao modelo antes da geração da resposta. Reduz alucinações e permite que o modelo responda com base em documentos internos e atualizados, sem precisar de re-treinamento (fine-tuning).

- **Agentes de IA e Uso de Ferramentas (Tool Use)**

Um agente de IA é um LLM orquestrado em um laço que decide quais ferramentas externas chamar (busca, calculadora, API, banco de dados), interpreta o resultado e decide o próximo passo, até concluir a tarefa ou atingir um critério de parada. Arquiteturas multiagente dividem uma tarefa complexa entre agentes especializados que colaboram (por exemplo, um agente planejador e outros agentes executores).

- **Matriz de Confusão e Métricas de Classificação**

Para um classificador binário: Verdadeiro Positivo (VP), Falso Positivo (FP), Verdadeiro Negativo (VN) e Falso Negativo (FN). $\text{Acurácia} = (\text{VP} + \text{VN}) / \text{Total}$ (enganosa em bases desbalanceadas). $\text{Precisão} = \text{VP} / (\text{VP} + \text{FP})$ (dos que o modelo previu positivo, quantos eram de fato). $\text{Recall/Sensibilidade} = \text{VP} / (\text{VP} + \text{FN})$ (dos que eram de fato positivo, quantos o modelo capturou). $\text{F1-Score} = \text{média harmônica entre Precisão e Recall}$, útil quando as duas importam e a base é desbalanceada.

- **Curva ROC e AUC**

A curva ROC (Receiver Operating Characteristic) plota a Taxa de Verdadeiros Positivos contra a Taxa de Falsos Positivos em diferentes limiares de decisão. A AUC (Area Under the Curve) resume essa curva em um único número entre 0,5 (modelo aleatório) e 1,0 (classificador perfeito), sendo mais robusta que a acurácia para comparar modelos em bases desbalanceadas.

ALERTA DE PROVA - PRECISÃO VERSUS RECALL

Priorize PRECISÃO quando o custo de um Falso Positivo for alto (por exemplo, acusar um contribuinte idôneo de sonegação). Priorize RECALL quando o custo de um Falso Negativo for alto (por exemplo, deixar passar um caso real de fraude fiscal)!

21. Referências e Conferência

Conferência realizada em 21/09/2026.

- Referências

Russell e Norvig, Artificial Intelligence: A Modern Approach; Goodfellow, Bengio e Courville, Deep Learning; Hastie, Tibshirani e Friedman, The Elements of Statistical Learning e ISLR; Géron, Hands-On Machine Learning; Vaswani et al., Attention Is All You Need (2017); Lewis et al. (RAG, 2020); Hu et al. (LoRA, 2021); Yao et al. (ReAct, 2022); documentação do Apache Hadoop, Apache Spark, scikit-learn, pandas e MLflow; NIST AI RMF 1.0; ISO/IEC 42001:2023; Lei nº 13.709/2018 (art. 20); Edital SEFAZ-CE 2026 (FCC), Anexo VI.

- Corrigido nesta revisão

Fórmulas em LaTeX (apareciam literais no PDF) reescritas em texto; "70% a 80% do esforço" atribuído com ressalva; "obrigatoriamente padronizadas" para redes neurais reformulado; DBSCAN "imune" reformulado com limitações; Spark "até 100 vezes" atribuído aos benchmarks do projeto e Mesos marcado como depreciado; embeddings do BERT diferenciados dos estáticos; lematização de "melhores"; C4.5 usa razão de ganho; Random Forest "sem aumentar o viés" suavizado; Porter é para inglês; seção de armadilhas atribuída às bancas sem evidência.

- A conferir

Situação do Projeto de Lei nº 2.338/2023 e do MCP; versões e recursos das bibliotecas; valores de referência de benchmarks; exemplos de plataformas de nuvem citados.

REGRA DE CONTEÚDO

Este material não afirma nada sem fonte. Se houver divergência com a documentação oficial, ela prevalece.

GLOSSÁRIO DE SIGLAS E ABREVIATURAS

Consulte abaixo o significado e a expansão explicativa de todas as siglas contábeis, tributárias e de TI presentes nas apostilas de preparação:

ACID	Atomicidade, Consistência, Isolamento e Durabilidade. Propriedades fundamentais para transações em SGBDs.
BI	Business Intelligence. Inteligência de Negócios para mineração e análise estratégica de dados.
CAP	Consistência, Disponibilidade e Tolerância a Partições. Teorema limitador para sistemas distribuídos.
CEBRASPE	Centro de Seleção e de Promoção de Eventos. Banca examinadora oficial de concursos públicos.
CFC	Conselho Federal de Contabilidade. Órgão responsável pela edição das NBCs no território nacional.
CIAP	Controle de Crédito de ICMS do Ativo Permanente. Controle fiscal de apropriação de créditos de bens imobilizados.
CID	Confidencialidade, Integridade e Disponibilidade. Pilares de sustentação da Segurança da Informação.
COBIT	Control Objectives for Information and Related Technologies. Framework global de Governança de TI.
CPC	Comitê de Pronunciamentos Contábeis. Emissor de normas convergentes com padrões internacionais.
FGV	Fundação Getulio Vargas. Banca examinadora de certames e instituição de pesquisa econômica.
ICMS	Imposto sobre Operações Relativas à Circulação de Mercadorias e Prestações de Serviços de Transporte.
IPVA	Imposto sobre a Propriedade de Veículos Automotores. Tributo de competência estadual.
ITD	Imposto sobre Transmissão Causa Mortis e Doação. Tributo sobre heranças e doações na Bahia.
ITIL	Information Technology Infrastructure Library. Boas práticas de Gerenciamento de Serviços de TI.
KDD	Knowledge Discovery in Databases. Processo sistemático de descoberta de conhecimento em bases de dados.
LDO	Lei de Diretrizes Orçamentárias. Define as metas e prioridades para o orçamento do exercício seguinte.
LOA	Lei Orçamentária Anual. Estima as receitas e fixa as despesas públicas para o exercício financeiro.
NBC TA	Normas Brasileiras de Contabilidade Técnicas de Auditoria Independente. Alinhadas ao padrão internacional.
NBC TI	Normas Brasileiras de Contabilidade Técnicas de Auditoria Interna. Foco nos processos organizacionais.
NoSQL	Not Only SQL. Família de SGBDs não relacionais estruturados para alto desempenho e escalabilidade.
OLAP	Online Analytical Processing. Ferramentas analíticas baseadas em cubos de dados multidimensionais.
PAF-BA	Processo Administrativo Fiscal do Estado da Bahia. Lei nº 3.956/1981 que regula o contencioso.
PPA	Plano Plurianual. Planejamento estratégico governamental de médio prazo (4 anos).
RICMS-BA	Regulamento do ICMS do Estado da Bahia. Decreto regulamentador do principal imposto estadual.
RUP	Rational Unified Process. Processo de desenvolvimento de software disciplinado e preditivo.
SEFAZ-BA	Secretaria da Fazenda do Estado da Bahia. Órgão de arrecadação e controle fiscal baiano.

SGBD	Sistema de Gerenciamento de Banco de Dados. Software de controle e armazenamento seguro de dados.
SQL	Structured Query Language. Linguagem declarativa padrão para consulta em SGBDs relacionais.
TI	Tecnologia da Informação. Recursos de hardware, software e infraestrutura de rede corporativa.
XP	Extreme Programming. Metodologia ágil de engenharia de software focada em excelência técnica.